ОТЧЁТ

по лабораторной работе №8

«Паттерны проектирования в Python»

Вариант 20. Интернет-провайдер

1. Цель работы

Научиться заменять жёсткие ветвления паттернами Factory, Strategy, Repository и Observer на примере тарифов интернет-провайдера.

2. Постановка индивидуального задания

Вариант 20. Система управления тарифами. Фабрика создаёт домашний, корпоративный и мобильный тариф. Стратегии считают абонентскую плату, скидку и дополнительные услуги. Дополнительный паттерн — Observer. Обязателен и репозиторий.

В проекте оставлен короткий наивный расчёт на словарях и if, а рабочее меню использует паттерны. Для репозитория есть память и JSON-реализация. Поведение проверяют тесты.

3. Краткие теоретические сведения

Фабрика прячет правила создания объектов: вызывающий код просит «домашний тариф», а не собирает его поля сам. Стратегия выносит алгоритм в отдельный объект, чтобы расчёт можно было менять без правки клиента. Репозиторий отделяет хранение от предметных операций. Наблюдатель подписывается на событие и узнаёт об изменении, не будучи вшитым в сам тариф.

4. Описание программы

TariffFactory по виду тарифа задаёт базовую плату. PriceCalculator складывает базовую плату, скидку и стоимость услуг. Скидка корпоративного тарифа — 10 процентов, мобильного — 5. TariffSubject рассылает сообщение всем подписчикам при создании тарифа и при смене платы.

MemoryTariffRepository хранит объекты в списке. JsonTariffRepository показывает вторую реализацию того же набора операций. Тесты проверяют фабрику, формулу корпоративного тарифа с двумя услугами, поиск в репозитории, два сообщения наблюдателя и отказ неизвестного вида.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — main.py

```
"""
Лабораторная работа №8. Вариант 20.
Паттерны: Factory, Strategy, Repository, Observer.
"""


import os
import sys

from naive import create_tariff, month_price
from observer import SubscriberNotifier, TariffSubject
from repository import MemoryTariffRepository
from service import TariffService

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")

BASE = os.path.dirname(os.path.abspath(__file__))



def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()



def read_int(prompt):
    while True:
        text = read_line(prompt)
        if text.isdigit():
            return int(text)
        print("Ошибка: введите целое неотрицательное число.")



def read_float(prompt):
    while True:
        text = read_line(prompt).replace(",", ".", 1)
        try:
            value = float(text)
        except ValueError:
            print("Ошибка: введите число.")
            continue
        if value <= 0:
            print("Ошибка: плата должна быть больше нуля.")
            continue
        return value



def print_menu():
    print()
    print("===== Тарифы провайдера =====")
    print("1. Показать наивный расчёт")
    print("2. Создать тариф (Factory)")
    print("3. Рассчитать стоимость (Strategy)")
    print("4. Изменить абонентскую плату (Observer)")
    print("5. Показать тарифы (Repository)")
    print("0. Выход")
```

```
    print("Виды: home, corp, mobile")
    print()


def main():
    subject = TariffSubject()
    subject.attach(SubscriberNotifier())
    service = TariffService(MemoryTariffRepository(), subject)
    print("Репозиторий в памяти. Наблюдатель подключён.")

    while True:
        print_menu()
        choice = read_line("Выберите действие: ")
        try:
            if choice == "1":
                raw = create_tariff("corp", "Старый корпоративный")
                print("Наивная ветка if:", raw)
                print("Цена с 2 услугами:", month_price(raw, 2))
                print("Новый вид тарифа в наивной версии требует правки функции create_tariff.")
            elif choice == "2":
                kind = read_line("Вид тарифа: ")
                name = read_line("Название: ")
                tariff = service.create(kind, name)
                print("Создан", tariff.title())
            elif choice == "3":
                name = read_line("Название тарифа: ")
                extras = read_int("Число дополнительных услуг: ")
                total = service.quote(name, extras)
                print(f"Итог: абонплата − скидка + услуги = {total:.0f} руб.")
            elif choice == "4":
                name = read_line("Название тарифа: ")
                fee = read_float("Новая абонентская плата: ")
                service.change_fee(name, fee)
            elif choice == "5":
                items = service.repository.all()
                if not items:
                    print("Тарифов пока нет.")
                for item in items:
                    print(" -", item.title())
            elif choice == "0":
                print("Работа программы завершена.")
                break
            else:
                print("Ошибка: выберите пункт от 0 до 5.")
        except ValueError as exc:
            print(f"Ошибка: {exc}")
        except EOFError:
            break


if __name__ == "__main__":
    main()
```

Листинг — factory.py

```
"""Фабрика семейства тарифов."""

from models import Tariff


class TariffFactory:
    def create(self, kind: str, name: str) -> Tariff:
        fees = {"home": 500.0, "corp": 2500.0, "mobile": 350.0}
        if kind not in fees:
            raise ValueError("неизвестный вид тарифа")
        return Tariff(kind, name, fees[kind])
```

Листинг — models.py

```
"""Тарифы, которые создаёт фабрика."""

from dataclasses import dataclass, field
```

```
@dataclass
class Tariff:
    kind: str
    name: str
    base_fee: float
    subscribers: list[str] = field(default_factory=list)

    def title(self) -> str:
        return f"{self.kind}: {self.name} ({self.base_fee:.0f} руб.)"
```

Листинг — naive.py

```
"""Наивный расчёт без паттернов: ветки if и прямое создание объектов."""



def create_tariff(kind, name):
    if kind == "home":
        return {"kind": kind, "name": name, "fee": 500, "discount": 0}
    if kind == "corp":
        return {"kind": kind, "name": name, "fee": 2500, "discount": 0.1}
    if kind == "mobile":
        return {"kind": kind, "name": name, "fee": 350, "discount": 0}
    raise ValueError("неизвестный вид")



def month_price(tariff, extras):
    fee = tariff["fee"]
    discount = tariff["fee"] * tariff["discount"]
    extra = 100 * extras
    return fee - discount + extra
```

Листинг — observer.py

```
"""Наблюдатели узнают об изменении тарифа."""

from typing import Protocol



class TariffObserver(Protocol):
    def update(self, message: str) -> None:
        ...



class SubscriberNotifier:
    def __init__(self):
        self.messages: list[str] = []

    def update(self, message: str) -> None:
        self.messages.append(message)
        print("Наблюдатель:", message)



class TariffSubject:
    def __init__(self):
        self._observers: list = []

    def attach(self, observer) -> None:
        self._observers.append(observer)

    def notify(self, message: str) -> None:
        for observer in self._observers:
            observer.update(message)
```

Листинг — repository.py

```
"""Интерфейс репозитория и две реализации: память и JSON-файл."""

import json
import os

from models import Tariff
```

```
class MemoryTariffRepository:
    def __init__(self):
        self._items: list[Tariff] = []

    def add(self, tariff: Tariff) -> None:
        self._items.append(tariff)

    def all(self) -> list[Tariff]:
        return list(self._items)

    def find(self, name: str) -> Tariff | None:
        for item in self._items:
            if item.name.lower() == name.lower():
                return item
        return None


class JsonTariffRepository:
    def __init__(self, path: str):
        self.path = path
        self._items: list[Tariff] = []
        self.load()

    def add(self, tariff: Tariff) -> None:
        self._items.append(tariff)
        self.save()

    def all(self) -> list[Tariff]:
        return list(self._items)

    def find(self, name: str) -> Tariff | None:
        for item in self._items:
            if item.name.lower() == name.lower():
                return item
        return None

    def save(self) -> None:
        folder = os.path.dirname(self.path)
        if folder:
            os.makedirs(folder, exist_ok=True)
        payload = [
            {"kind": item.kind, "name": item.name, "base_fee": item.base_fee}
            for item in self._items
        ]
        with open(self.path, "w", encoding="utf-8") as file:
            json.dump(payload, file, ensure_ascii=False, indent=2)

    def load(self) -> None:
        if not os.path.exists(self.path):
            self._items = []
            return
        with open(self.path, "r", encoding="utf-8") as file:
            raw = json.load(file)
        self._items = [Tariff(item["kind"], item["name"], item["base_fee"]) for item in raw]
```

Листинг — service.py

```
"""Сборка фабрики, стратегий, репозитория и наблюдателя."""

from factory import TariffFactory
from models import Tariff
from observer import TariffSubject
from strategies import DiscountStrategy, ExtraServicesStrategy, PriceCalculator, BaseFeeStrategy


class TariffService:
    def __init__(self, repository, subject: TariffSubject):
        self.repository = repository
        self.factory = TariffFactory()
        self.subject = subject
```

```
    def create(self, kind: str, name: str) -> Tariff:
        tariff = self.factory.create(kind, name)
        self.repository.add(tariff)
        self.subject.notify(f"создан тариф «{name}» ({kind})")
        return tariff

    def quote(self, name: str, extra_count: int) -> float:
        tariff = self.repository.find(name)
        if tariff is None:
            raise ValueError("тариф не найден")
        calculator = PriceCalculator(
            [
                BaseFeeStrategy(),
                DiscountStrategy({"corp": 0.1, "home": 0.0, "mobile": 0.05}),
                ExtraServicesStrategy(100, extra_count),
            ]
        )
        return calculator.total(tariff)

    def change_fee(self, name: str, new_fee: float) -> None:
        tariff = self.repository.find(name)
        if tariff is None:
            raise ValueError("тариф не найден")
        tariff.base_fee = new_fee
        self.subject.notify(f"абонентская плата «{name}» изменена на {new_fee:.0f} руб.")
```

Листинг — strategies.py

```
"""Стратегии абонентской платы, скидки и дополнительных услуг."""

from typing import Protocol

from models import Tariff



class FeeStrategy(Protocol):
    def amount(self, tariff: Tariff) -> float:
        ...



class BaseFeeStrategy:
    def amount(self, tariff: Tariff) -> float:
        return tariff.base_fee



class DiscountStrategy:
    def __init__(self, rates: dict[str, float]):
        self.rates = rates

    def amount(self, tariff: Tariff) -> float:
        rate = self.rates.get(tariff.kind, 0.0)
        return tariff.base_fee * rate



class ExtraServicesStrategy:
    def __init__(self, price_per_service: float, count: int):
        self.price_per_service = price_per_service
        self.count = count

    def amount(self, tariff: Tariff) -> float:
        return self.price_per_service * self.count



class PriceCalculator:
    """Складывает выбранные стратегии. Новая стратегия не требует правки калькулятора."""

    def __init__(self, strategies: list):
        self.strategies = strategies

    def total(self, tariff: Tariff) -> float:
        fee = 0.0
```

```
        discount = 0.0
        extra = 0.0
        for strategy in self.strategies:
            value = strategy.amount(tariff)
            if isinstance(strategy, DiscountStrategy):
                discount += value
            elif isinstance(strategy, ExtraServicesStrategy):
                extra += value
            else:
                fee += value
        return fee - discount + extra
```

Листинг — tests\test_patterns.py

```
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from factory import TariffFactory
from models import Tariff
from observer import TariffSubject
from repository import MemoryTariffRepository
from service import TariffService
from strategies import BaseFeeStrategy, DiscountStrategy, ExtraServicesStrategy, PriceCalculator



class MemoryObserver:
    def __init__(self):
        self.messages = []

    def update(self, message: str) -> None:
        self.messages.append(message)



def test_factory_kinds():
    factory = TariffFactory()
    assert factory.create("home", "Дом").base_fee == 500
    assert factory.create("corp", "Офис").base_fee == 2500
    assert factory.create("mobile", "Мобайл").base_fee == 350



def test_strategy_corporate_discount_and_extras():
    tariff = Tariff("corp", "Офис", 2500)
    calculator = PriceCalculator(
        [BaseFeeStrategy(), DiscountStrategy({"corp": 0.1}), ExtraServicesStrategy(100, 2)]
    )
    assert calculator.total(tariff) == 2450



def test_repository_find():
    repo = MemoryTariffRepository()
    repo.add(Tariff("home", "Дом", 500))
    assert repo.find("дом").kind == "home"
    assert repo.find("нет") is None



def test_observer_on_fee_change():
    observer = MemoryObserver()
    subject = TariffSubject()
    subject.attach(observer)
    service = TariffService(MemoryTariffRepository(), subject)
    service.create("mobile", "Мобайл")
    service.change_fee("Мобайл", 400)
    assert len(observer.messages) == 2
    assert "400" in observer.messages[1]



def test_unknown_kind():
    try:
        TariffFactory().create("vip", "VIP")
        assert False
```

```
except ValueError:
    assert True
```

6. Примеры работы программы

Снимки сравнивают наивный расчёт с фабрикой, показывают стратегии и наблюдателя, а также результат тестов.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python8> python main.py

Репозиторий в памяти. Наблюдатель подключён.

===== Тарифы провайдера =====
1. Показать наивный расчёт
2. Создать тариф (Factory)
3. Рассчитать стоимость (Strategy)
4. Изменить абонентскую плату (Observer)
5. Показать тарифы (Repository)
0. Выход
Виды: home, corp, mobile

Выберите действие: 1
Наивная ветка if: {'kind': 'corp', 'name': 'Старый корпоративный', 'fee': 2500, 'discount': 0.1}
Цена с 2 услугами: 2450.0
Новый вид тарифа в наивной версии требует правки функции create_tariff.

===== Тарифы провайдера =====
1. Показать наивный расчёт
2. Создать тариф (Factory)
3. Рассчитать стоимость (Strategy)
4. Изменить абонентскую плату (Observer)
5. Показать тарифы (Repository)
0. Выход
Виды: home, corp, mobile

Выберите действие: 2
Вид тарифа: home
Название: Домашний 100
Наблюдатель: создан тариф «Домашний 100» (home)
Создан home: Домашний 100 (500 руб.)

===== Тарифы провайдера =====
1. Показать наивный расчёт
2. Создать тариф (Factory)
3. Рассчитать стоимость (Strategy)
4. Изменить абонентскую плату (Observer)
5. Показать тарифы (Repository)
0. Выход
Виды: home, corp, mobile

Выберите действие: 2
Вид тарифа: corp
Название: Офис 500
Наблюдатель: создан тариф «Офис 500» (corp)
Создан corp: Офис 500 (2500 руб.)

===== Тарифы провайдера =====
1. Показать наивный расчёт
2. Создать тариф (Factory)
3. Рассчитать стоимость (Strategy)
4. Изменить абонентскую плату (Observer)
5. Показать тарифы (Repository)
0. Выход
Виды: home, corp, mobile

Выберите действие: 2
Вид тарифа: mobile
Название: Мобильный
Наблюдатель: создан тариф «Мобильный» (mobile)
Создан mobile: Мобильный (350 руб.)

===== Тарифы провайдера =====
1. Показать наивный расчёт
2. Создать тариф (Factory)
3. Рассчитать стоимость (Strategy)
4. Изменить абонентскую плату (Observer)
5. Показать тарифы (Repository)
0. Выход
Виды: home, corp, mobile
```

Рисунок 1 – Наивный расчёт и создание тарифов через фабрику

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python8> python main.py

Репозиторий в памяти. Наблюдатель подключён.

===== Тарифы провайдера =====
1. Показать наивный расчёт
2. Создать тариф (Factory)
3. Рассчитать стоимость (Strategy)
4. Изменить абонентскую плату (Observer)
5. Показать тарифы (Repository)
0. Выход
Виды: home, corp, mobile

Выберите действие: 2
Вид тарифа: corp
Название: Офис 500
Наблюдатель: создан тариф «Офис 500» (corp)
Создан corp: Офис 500 (2500 руб.)

===== Тарифы провайдера =====
1. Показать наивный расчёт
2. Создать тариф (Factory)
3. Рассчитать стоимость (Strategy)
4. Изменить абонентскую плату (Observer)
5. Показать тарифы (Repository)
0. Выход
Виды: home, corp, mobile

Выберите действие: 2
Вид тарифа: mobile
Название: Мобильный
Наблюдатель: создан тариф «Мобильный» (mobile)
Создан mobile: Мобильный (350 руб.)

===== Тарифы провайдера =====
1. Показать наивный расчёт
2. Создать тариф (Factory)
3. Рассчитать стоимость (Strategy)
4. Изменить абонентскую плату (Observer)
5. Показать тарифы (Repository)
0. Выход
Виды: home, corp, mobile

Выберите действие: 3
Название тарифа: Офис 500
Число дополнительных услуг: 2
Итог: абонплата − скидка + услуги = 2450 руб.
```

Рисунок 2 – Стратегии цены и уведомление наблюдателя об изменении платы

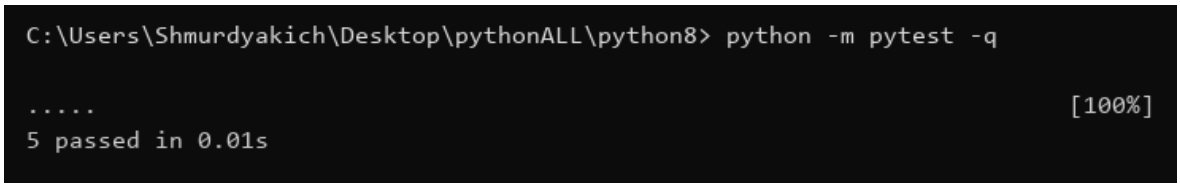


Рисунок 3 – Тесты фабрики, стратегии, репозитория и наблюдателя

7. Ответы на контрольные вопросы

1. Зачем фабрика, если можно вызвать конструктор?

Фабрика собирает объект по виду тарифа и не заставляет меню знать базовые цены. Новый вид добавляется в одном месте.

2. Чем стратегия лучше ветки if?

Каждый способ расчёта — отдельный класс. Калькулятор их комбинирует, а не обрастает условиями.

3. Что делает наблюдатель в этой работе?

При создании тарифа и при смене абонентской платы подписчик получает сообщение. Тариф сам не печатает уведомление.

4. Зачем две реализации репозитория?

Сервис работает с одним и тем же набором методов. Память удобна для демонстрации, JSON сохраняет тарифы между запусками.

5. Какая проблема была у наивной функции?

Новый вид тарифа и новое правило скидки требовали править ту же функцию. Паттерны убирают эту правку из клиентского кода.

8. Вывод

В лабораторной работе №8 тарифы варианта 20 создаются фабрикой, цена собирается из стратегий, хранение спрятано за репозиторием, а изменение платы рассылается наблюдателям. Наивный вариант оставлен в отчёте как исходная точка, тесты фиксируют итоговые правила расчёта.